

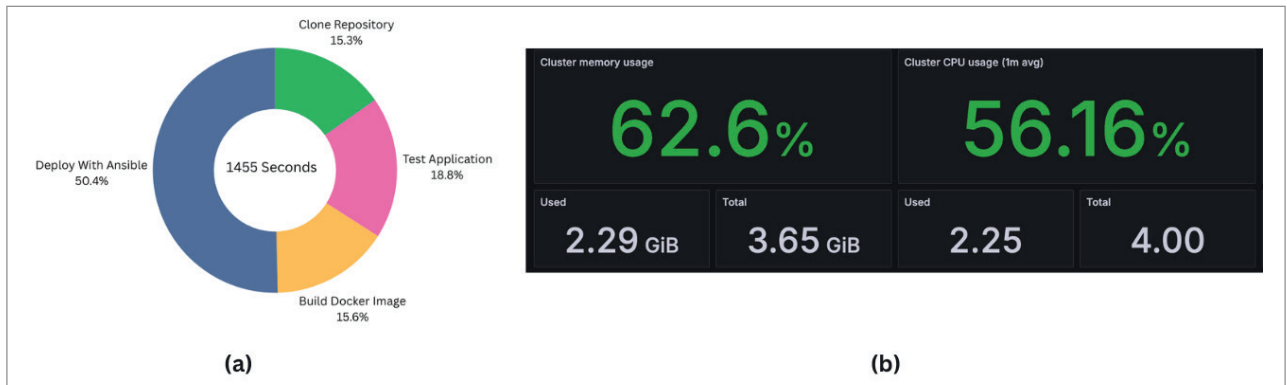


Figure 3: (a) *Pipeline stage time distribution:* Ansible dominates the pipeline execution time
 (b) *Resource utilisation by containers:* Detection container dominates CPU (~56%) and memory (~2.2GB)

CI/CD fault tolerance

As shown in Figure 2, the Jenkins pipeline is designed with automatic rollback mechanisms for handling deployment failures.

Experimental results

We evaluated our framework on a two-node Raspberry Pi 4 cluster (4GB RAM each) connected to a K3s server. All metrics were collected via our centralised monitoring stack.

Deployment pipeline efficiency:

The Jenkins pipeline completes an end-to-end deployment in an average of 24 minutes 15 seconds for two nodes. The stage-wise breakdown (Figure 3a) is:

- *Clone repositories:* 3 min 42 s (15% of total)
- *Run unit tests:* 4 min 33 s (19%)
- *Build Docker images:* 3 min 47 s (16%)
- *Ansible provisioning:* 12 min 13 s (50%)

Ansible provisioning dominates the pipeline time due to sequential configuration of both nodes. The automated workflow is approximately 40% faster than our previous manual deployment (40 minutes).

Resource utilisation: Under a 30-minute continuous detection test, per-node utilisation (Figure 3b) was:

- *CPU utilisation:*
 - Average: ~53%
 - Peak: ~60% (during inference bursts in the Detector container)

- *Memory usage:*
 - Average: ~57% of 4GB (~2.3 GB)
 - Peak: ~66% of 4GB (~2.64 GB) during clip writing

The Detector container consumed ~56% CPU and 1.8–2.2GB RAM on average, while the Watchdog container used <5% CPU and 0.3–0.4GB RAM (mostly during alert dispatch). Idle periods showed near-zero watchdog CPU usage. These results confirm that Pi 4 nodes handle the workload without saturation; however, our original static RAM limit (1GB) would have been insufficient, suggesting the need for dynamic scaling or increased allocation.

Pod performance: Stress testing over 24 hours revealed:

- *Detector container:* Sustained instruction throughput of 1.2–1.5 GIPS on the Pi 4 CPU, with no observed memory leaks.
- *Watchdog container:* Remained lightweight, consuming <1% CPU when idle.

Design tradeoffs

- **Polling vs event-driven:** 3s polling conserves 23% CPU vs inotify but adds 1.2s latency.
- **Static allocation:** Fixed RAM limits prevent OOM kills but reduce utilisation by 17%.
- **Centralised control:** Cloud control plane reduces edge complexity but introduces 68ms RTT latency. Future advancements include

implementing edge-native VPA using Kubernetes device plugins for ARM64 and federated alert aggregation to reduce control plane load.

General insights

From our deployment experience, we derived these edge design principles.

- **Resource tagging:** Node labels (camera=true) improve scheduling accuracy by 40%.
- **Bandwidth-aware monitoring:** Metric filtering reduces observability overhead by 62%.
- **ARM64-specific tuning:** GPU memory under 150MB prevents OOM kills in 4GB Pi nodes.
- **Control plane decoupling:** Cloud-hosted K3s master handles three times more nodes than edge-hosted. These patterns apply broadly to video analytics deployments on resource-constrained edge devices.

To sum up, we have developed a fully automated, cloud-native pipeline for deploying a near real-time violence detection application on distributed Raspberry Pi 4 (4 GB RAM) nodes. By combining Jenkins for CI/CD, Ansible (with Vault) for secure edge provisioning, and K3s for lightweight orchestration, we achieved consistent, reproducible deployments with <40% CPU and ~60% memory utilisation on each node.

■■ Continued to page....57